applied sciences

MDPI

Article

ESL: A High-Performance Skiplist with Express Lane

Yedam Na [1], Bonmoo Koo [1], Taeyoon Park [2], Jonghyeok Park [3] and Wook-Hee Kim [1,*]

[1] Department of Computer Science and Engineering, Konkuk University, Seoul 05029, Republic of Korea
[2] Department of Smart ICT Convergence, Konkuk University, Seoul 05029, Republic of Korea
[3] Division of Computer Engineering, Hankuk University of Foreign Studies, Yongin 17035, Republic of Korea
* Correspondence: wookhee@konkuk.ac.kr

Abstract: With the increasing capacity and cost-efficiency of DRAM in multi-core environments, in-memory databases have emerged as fundamental solutions for delivering high performance. The index structure is a crucial component of the in-memory database, which, leveraging fast access to DRAM, plays an important role in the performance improvement and scalability of in-memory databases. A skiplist is one of the most widely used in-memory index structures and it has been adopted by popular databases. However, skiplists suffer from poor performance due to their structural limitations. In this work, we propose ESL, a high-performance and scalable skiplist. ESL efficiently enhances the performance of traverse operations by optimizing index levels for the CPU cache. With CPU cache-optimized index levels, we synergistically leverage a combination of exponential and linear searches. In addition, ESL reduces synchronization overhead by updating the index levels asynchronously, while tolerating inconsistencies. In our YCSB evaluation, ESL improves throughput by up to 2.8× over other skiplists in high-level evaluations. ESL also shows lower tail latency than other skiplists by up to 35×. Also, ESL consistently shows higher throughput in our real-world workload evaluation.

Keywords: skiplist; in-memory data structure; in-memory database; scalability; index structure

check for updates

Citation: Na, Y.; Koo, B.; Park, T.; Park, J.; Kim, W.-H. ESL: A High-Performance Skiplist with Express Lane. *Appl. Sci.* **2023**, *13*, 9925. https://doi.org/10.3390/app13179925

Academic Editors: Tian-Fu Lee, I-Pin Chang and Tung-Kuan Liu

Received: 30 July 2023
Revised: 29 August 2023
Accepted: 31 August 2023
Published: 1 September 2023

Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Database systems are essential software systems that efficiently manage large amounts of data. They provide various functionalities for the user's convenience. Also, database systems provide strong data consistency guarantees by providing durability and crash consistency. However, these functionalities and crash consistency protocols can burden the performances of database systems [1–6].

The data amounts have been continuously increasing [7]. In-memory databases are widely used solutions because they provide high throughput with low latency. However, due to their high costs and physical capacity limitations, in-memory databases are focusing more on scalability and cost-efficiency. Recently, the cost of DRAM has decreased [8]. Moreover, emerging memory technologies, such as persistent memory [9] and CXL-based memory [10], are expected to increase the memory capacity.

The increasing number of cores and computational power in servers [11–14] presents an opportunity to improve the performance of in-memory databases. As most servers have multiple cores, in-memory databases can process the queries in a concurrent manner. Hence, hardware advancements offer the potential to synergistically improve the performance of in-memory databases by leveraging modern hardware.

In-memory databases mainly operate within DRAM-based memory spaces, making index structures play a key role in these databases. Index structures can be categorized as ordered indices and hash-based indices. Ordered indices offer moderate search performance but good range query performance. Hash indices provide high performance for pointer queries, yet they lag in terms of range query performance.

Tree-based index structures, such as B+-tree [15–18], are widely used as representative ordered indices. Tree-based index structures show good performance in both point queries and range queries. However, tree-based data structures require structure modification operation (SMO), and these operations can reduce the query performance due to their complexity. For instance, the split operation of the B+-tree creates a new node and moves half of the entries to the new node. The parent entry of the new node is added to the parent node. This split operation is repeated until the node has enough space.

By virtue of simplicity, the beauty of a skiplist [19] lies in its ability to offer a more straightforward and cost-effective SMO compared to a tree-based structure. There is no split and merge operation in a skiplist. Instead, it adds a new node to the existing ones by connecting the new node's next pointer to the previous nodes.

As a result, most LSM-tree-based databases, such as LevelDB [20], RocksDB [21], Hbase [22], and MongoDB [23], adopt a skiplist as the data structure for their MemTable. A skiplist is composed of multiple levels of linked lists, simplifying the SMOs. A skiplist generates multiple levels of each entry probabilistically and maintains them to reduce the cache misses when traversing the data structure. A skiplist can be divided into multiple levels, denoted as index levels and data levels. The index levels provide the shortest path to the target data node in the data level. While the multiple levels of each entry reduce cache misses, a skiplist can still suffer from performance degradation due to cache misses. Additionally, these multiple levels can extend the tail latencies of write queries, which include insert and delete operations; the insert and delete operations involve updates to the index level.

Several efforts were made to improve the performance of a skiplist. Cache-Sensitive Skiplist [24] merges upper-level entries into a single array. Parallel in-memory Skiplist [25] merges multiple entries of neighbors of the same level into the skiplist. No Hot Spot Skiplist [26] adopts asynchronous updates for the index levels of a skiplist to reduce the latency of the write operation. NUMASK [27] considers NUMA awareness by replicating index levels to each NUMA socket.

Previous studies focused on either reducing the length of the critical path or reducing the cache misses. However, we could not find the skiplist, which considers both reducing the length of the critical path and reducing the cache misses. However, to the best of our knowledge, previous studies required additional rebalancing operations to judiciously rebuild the index levels.

In this work, we propose a high-performance and scalable skiplist, denoted as the Express Skiplist (ESL). We aim to achieve superior performance for both read and write operations without any compromises, ensuring high scalability.

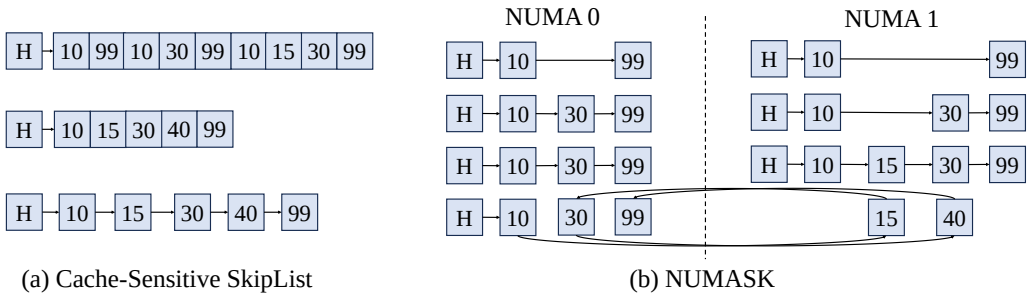
The contributions of the paper are as follows:

- We propose a new in-memory skiplist, denoted as ESL, which is composed of a cache-optimized index level (COIL), a link-based data level, and a parent of data level (PDL) between the COIL and the data level.
- We exploit the asynchronous update for the COIL and the PDL and update them using multiple background threads.
- We develop a technique to accelerate the read performance and tolerate inconsistencies. Our technique combines the exponential search [28] and linear search to accelerate the read operation and tolerate the inconsistencies in the COIL caused by the background thread shift operation.

The rest of the paper is organized as follows. In Section 2, we explain the background and motivation of this work. Section 3 describes the design overview of ESL. Section 4 presents the details of the design of ESL. Section 5 shows how we implement ESL. Section 6 shows the performance evaluation and experimental analysis of ESL. We discuss the limitations of ESL and a skiplist for emerging storage devices in Section 7; we conclude the paper in Section 8.

2. Background

In this section, we explain the background of this work. First, we provide a brief introduction of several variants of skiplists and introduce the concept of endurable transient inconsistency for designing scalable data structures (Figure 1).



(a) Cache-Sensitive SkipList (b) NUMASK

Figure 1. Skiplist structures.

2.1. Skiplist

A skiplist [19] is one of the most widely used data structures in storage systems, such as RocksDB [21], LevelDB [20], Hbase [22], and so on. A skiplist is a scalable data structure that is composed of multiple levels. In a skiplist, a higher level is a sparse-linked list, as it provides opportunities to skip nodes in the lower level. A skiplist can be divided into two parts, index levels and data levels. The data level is the lowest skiplist level and contains all of the data of the skiplist. Index levels contain all of the skiplist levels, except for the data level, and provide the shortest path to the target data node in the data level. A skiplist does not need complicated structure-modified operations (SMO), such as split and merge operations in tree-based data structures.

A skiplist may fall short of the expected performance because it suffers from performance degradation from cache misses in linked-list-based structures. Also, adding more key–value pairs to additional index levels results in overhead and extended tail latency. There are multiple variants of skiplists [24–27] aimed at overcoming these main limitations of a skiplist.

2.1.1. Cache-Conscious Skiplist

Some research efforts were made to fundamentally reduce cache misses in a skiplist structure. In the following works, researchers focused on reducing cache misses by merging multiple nodes.

Cache-Sensitive Skiplist (CSSL) [24] is a variant of a skiplist, which reduces the number of cache misses by merging the multiple nodes in the index levels into a single array, denoted as a *fast lane*. The fast lane is a consecutive array that contains all nodes, with the exception of the data level, which contains all data in the skiplist. As the nodes are in a single array and stored in a sorted manner, the data structure is more cache-friendly. CSSL includes another array, denoted as the *proxy lane*, which consists of the data stored at the data level. Since the proxy lane is the array, it accelerates the process of finding a target data node not found at higher levels. However, because the nodes of both index levels and data levels in a skiplist are stored in separate arrays, numerous shift operations are required for node insertion or deletion. As shown in [24], CSSL delays the fast lane and proxy lane updates to mitigate the shift operation overhead. However, it leads to the periodic reconstruction of the fast lane and proxy lane, which may hinder overall performance.

Parallel in-memory Skiplist (PSL) [25] is a skiplist that merges multiple nodes into a node that is similar to the B+-tree node. This structure enables reducing the number of cache misses in the traversing skiplist. The Parallel in-memory Skiplist also defers updates of higher index levels and rebuilds the structure when the number of updates reaches the threshold using background threads. Since it periodically rebuilds the whole index structure in an asynchronous manner, the background threads can be the performance bottleneck.

2.1.2. Asynchronous Update for Index Levels

A skiplist, with its simpler structure compared to a tree-based structure, ensures that its SMO is relatively cost-effective. However, the SMO entails overhead in the execution of each skiplist operation. In order to reduce the overhead along the critical path, previous studies have leveraged asynchronous updates.

No Hot Spot Skiplist (NHSSL) [26] decouples each level within the skiplist and updates the index levels asynchronously using background threads. The NHSSL hides the update overhead for index levels and employs the additional rebalancing operation to enhance the skiplist.

NUMASK [27] is an improved version of NHSSL, so it also exploits asynchronous updates for index levels. In addition, NUMASK considers NUMA awareness by replicating index levels for every NUMA socket. Also, NUMASK adds an additional level—denoted as the *intermediate layer*—to the index level to efficiently find the data, which are stored in the NUMA remote socket.

2.2. Tree-Based In-Memory Index Structure

Index structures are fundamental building blocks in the storage system. Numerous studies have aimed to improve the index structure performance. *CSB+-tree* [16] makes the B+-tree CPU cache-efficient by removing child pointers. Since the node only includes the keys, the read operation is more efficient. *FAST* [29] is an architecture-optimized index structure, tailored for modern CPUs and GPUs. CSB+-tree and FAST are designed for read-intensive workloads; they are not good for write-intensive workloads. ESL is designed for both read- and write-intensive workloads. *Blink-Hash* [30] employs a node structure, which is a combination of the hash table and a blink-tree to efficiently process the time-series workload with keys sorted in ascending order in ascending order. ESL is designed for the various workloads of key–value stores, which can be in random order. Recent studies were conducted on index structures for near-data processing. In particular, process-in-memory (PIM) has emerged as a promising near-data processing hardware. HybriDS [31] employs a hybrid approach, which combines a skiplist with B+-tree. PIM-tree [32] develops the push–pull search with a hybrid tree structure, which combines shadow subtrees and chunked skiplists. Also, the concept of the learned index [33] has been proposed. The basic idea of the learned index is to replace the internal nodes with models to find the target data nodes. Initially, the learned index did not support the write operation. ALEX [34] provides write operations for the learned index. PLIN [35] optimizes the learned index structure for non-volatile memory, and ROLEX [36] leverages the learned index for key–value stores for disaggregated memory.

2.3. Endurable Transient Inconsistency

Endurable transient inconsistency [17,37,38] is a concept used for designing highly scalable index structures in multi-core systems. The basic idea of endurable transient inconsistency revolves around the idea that readers can find the target data, even if the index structure is in a transient state. In the FastFair [17] B+-tree, the reader can detect transient inconsistency by finding duplicate pointers. They can determine the target leaf node by traversing, using the sibling pointer of the leaf node. HydraLIST [37] and PACTree [38] also traverse their leaf nodes when their internal nodes indicate the neighboring node of the target node. A skiplist is a data structure that is well-suited for adopting endurable transient inconsistency. This is because a skiplist can also find the target node by traversing the level, even if the upper levels indicate the neighboring node.

3. Design Overview

3.1. Design Goal of ESL

In order to design a high-performance and scalable skiplist structure, we aim to fulfill the following two design goals.

Multi-core scalability. Most computer systems, from embedded devices to servers in the data center, have multiple core numbers, so the skiplist structure should be able to efficiently exploit multiple cores.

Cache Efficiency Since the CPU caches have lower latency than the main memory, the key to improving the performance of a skiplist, such as the in-memory data structure, is to efficiently leverage the CPU caches.

3.2. Cache-Optimized Index Level (COIL) with Express Lane

The main reason for the huge performance degradation in the legacy skiplist is the number of cache misses encountered while traversing several linked list nodes. There is no guarantee that the neighboring nodes will be located closely; thus, moving to the next level results in another cache miss. To resolve the performance degradation from cache misses, we designed the *cache-optimized index level* (COIL). The basic idea of the COIL is to merge the nodes from the same level to a consecutive array. All levels are sorted and consecutively stored, so we leverage the exponential search to accelerate the read performance. The index level in the COIL that employs exponential search is denoted as the *express lane*. When ESL finds a larger key than the given key, ESL triggers a linear search to leverage the advantages of the CPU cache and prefetcher. To add/remove the new data in the levels, we leverage 8-byte atomic write operations for a shift operation similar to that of the failure-atomic shift operation [17].

The express lane is starkly different from the fast lane/proxy lane in CSSL as the express lane consists of multiple index levels, and it implements exponential search. Furthermore, the express lane is updated asynchronously, so it does not require an additional rebuilding process in CSSL.

3.3. Lock-Free Data Level and Parent of the Data Level (PDL)

The data level holds all data in the ESL. Also, all the foreground threads concurrently access this data level for reading or updating its contents. Hence, ESL adopts a lock-free linked list for the data level, ensuring scalable and concurrent access. Since the data are independently stored in a linked list node, they can provide fine-grained synchronization. Hence, the scalability of ESL can be enhanced. We also leverage a lock-free linked list for the *parent of the data level* (PDL), which is managed by the background thread. Since the lowest level of the index level would have a higher chance of being selected, there can be many data, which may lead to large shift operations. Thus, ESL reduces the shift operation overhead by adopting a lock-free linked list instead of an array.

3.4. Asynchronous Update of the COIL and the PDL

The shift operation would incur huge overhead when large numbers of data are stored at the index level. In a skiplist, the write operation should update the lowest-level linked list as well as the upper levels. The upper levels would be determined with probability. In this case, the shift operation would lengthen the elapsed time for the write operation. To shorten the elapsed time for the write operation, ESL exploits background threads to asynchronously update the upper levels. The foreground threads first traverse the upper levels and then find the target location at the data level. The foreground threads update the linked list and then write the operation log entry and insert it into the operation log. The background threads read the operation log entry and then conduct the operation for the given levels.

3.5. Synchronization

ESL is composed of three levels: the COIL, the data level, and the PDL. As mentioned in Section 3.4, we decouple these into three levels and asynchronously update the COIL and the PDL. Hence, ESL adopts two different synchronization strategies.

COIL: We adopt the *read-optimized write exclusive (ROWEX)* [39] protocol for the COIL. Traversing the COIL is an essential operation in ESL, as every operation has to find the target location using the COIL. The foreground threads do not modify the COIL; only the background threads update the COIL. Hence, ESL leverages the ROWEX protocol, which provides a non-blocking read operation with only a writer.

The data level and the PDL: ESL exploits the lock-free linked list for the data level and the PDL. The main advantage of using a lock-free linked list for the data level is that the foreground threads can read or update the data level simultaneously without blocking. Also, the lock-free linked list helps minimize the tail latency [40]. Since each key–value pair is stored in a single linked list node, in the critical section, ESL only has to conduct a pointer update operation to connect the newly created data node.

The PDL is also based on a lock-free linked list, so the multiple background threads can update simultaneously, and do not incur a shift operation. The PDL is located above the data level, so it has a higher chance to add new data. Thus, exploiting a lock-free linked list is helpful in reducing the background thread overhead.

Operation log: The operation log in ESL works in a lock-free manner. The operation log queue is implemented using *compare-and-swap (CAS)* instructions. The foreground threads first acquire the log space by increasing the *tail* of the operation log. The foreground threads can write the operation log entries only when they successfully update the allocation index number. Similarly, the background threads have to update the *head* using a CAS operation before processing the operation log entries.

4. Design of ESL

In this section, we propose a new skiplist, denoted as ESL, and present its overall structure, semantics, and concurrency control scheme.

4.1. Structure of ESL

Figure 2 presents the overall structure of ESL. ESL consists of the COIL, the data level, and the parent of the data level (PDL). The COIL is composed of arrays of nodes. The nodes in the same level are merged into the same array. Figure 3 describes the COIL structure. The data level and the PDL are the lock-free linked lists, as shown in Figure 4. The data level keeps all data from ESL. The PDL is the lowest index level. ESL adopts the PDL as the base level of its index levels to circumvent the shift operation (see Section 4.2). The background threads read the operation log entries from the operation log and add these entries to their respective index levels in an asynchronous manner.

4.2. Write Operation (Insert/Update/Delete)

The write operation in ESL first traverses the COIL and then finds the target locations in the PDL and the data level. In the PDL and the data level, the writer performs the write operation in a lock-free manner and writes the operation log entry.

Insert. Since a skiplist is a probabilistic data structure, it needs to generate a random number to determine the level for a given key–value pair. When the level is determined, the writer allocates a new data node at the data level and writes the key–value pair. After allocating the data node, the writer traverses the COIL (❶ in Figure 2) and the PDL, (❷), finds the target location at the data level (❸ in Figure 2), and then adds it using the CAS instruction to atomically connect the newly created data node (❹ in Figure 2). Finally, the writer creates a new operation log entry and adds it to the operation log (❺ in Figure 2). The background threads will process the operation using the operation log in an asynchronous manner.

Update. The update operation of ESL does not create a new key–value pair. Instead, it changes the pointer value in the existing data node at the data level, which has the same key. Hence, the update operation of ESL requires traversing the COIL, the PDL, and the data level. Within the data level, the writer finds the target data node. After finding the target data node, the writer changes the pointer value atomically using the CAS instruction. In the update operation, ESL does not employ an asynchronous update for this update operation since there is no update operation for the COIL.

Delete. ESL's delete operation traverses the COIL and the PDL, similar to the insert and update operation. If the writer finds the index node, which has the same key as the given key, the writer logically deletes the index node by marking it as a deleted node. After marking the node, the writer moves to the lower level and logically deletes the index node at the index level. At the PDL and the data level, the writer finds the target node and deletes the data node by disconnecting the pointer from the previous node. After disconnecting the pointer, the writer deletes the data node. In order to safely reclaim the memory space, ESL uses epoch-based memory reclamation [37,38,41,42]. Note that since the index node in the lowest level of the COIL and the PDL has the target node's virtual address instead of the index, deleting the data node from the PDL does not hurt the correctness.

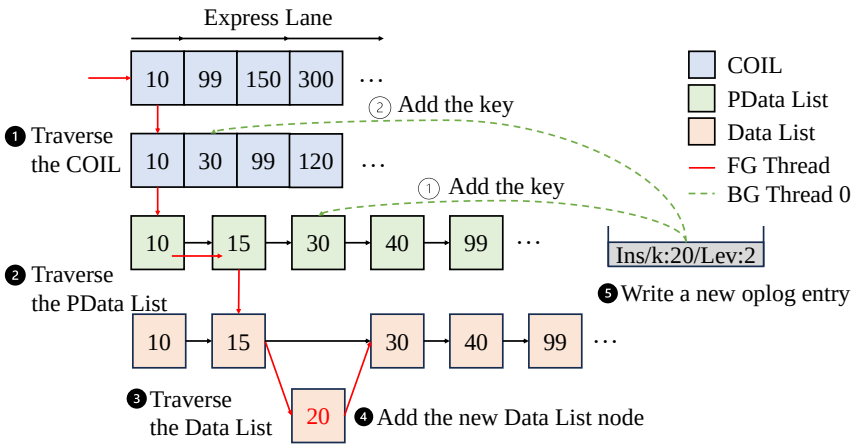


Figure 2. An illustration of the ESL structure and an example of the insert operation. ❶ The foreground thread traverses the COIL to find the target node in the PDL. ❷ The foreground thread finds the target data node by traversing the PDL. ❸ The foreground thread moves to the target data node at the data level. ❹ The foreground thread adds the new data node using a CAS operation. ❺ The foreground thread writes the operation log entry. ①, ② The background thread reads the operation log entry from the operation log and adds the new data to the index level according to the operation log entry.

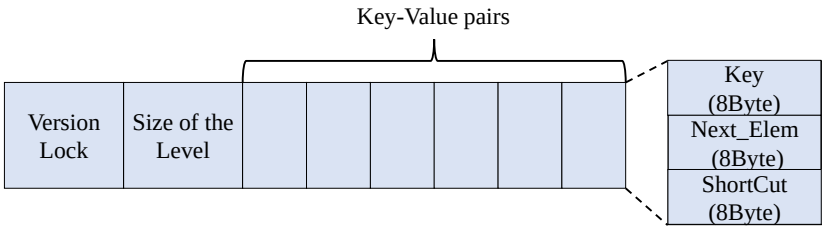


Figure 3. Layout of index levels in the COIL. Each entry of an index level consists of an 8-byte key, 8-byte next_elem, an index of the key at the next level, and an 8-byte shortcut. next_elem is an index number at the next level. The shortcut is a pointer to the data node.

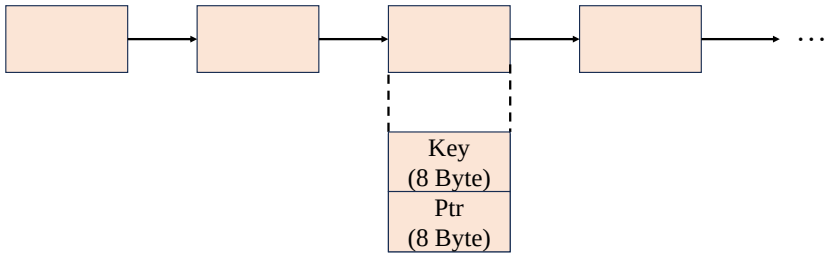


Figure 4. Layout of linked lists used in the PDL and the data level. Each node has an 8-byte key and an 8-byte value.

4.3. Asynchronous Update

ESL updates the PDL and the COIL using single or multiple background threads (①, ② in Figure 2). The asynchronous updates are invoked by an insert operation to add an entry to the index levels. The insert operations for the PDL and the COIL are shown in Figure 5. Updating the COIL entails shift operation; thus, the performance degradation is proportional to the number of shift operations. Figure 6 shows how the background thread updates the COIL. In order to make the reader tolerable to the transient inconsistency, the background thread writes the data to the index level in order. First, the background thread reserves a space for storing a new entry (①, ②, ③ in Figure 6, so it shifts the keys, as shown in Figure 6. The background thread writes the *next_elem* (④) and then writes the shortcut address (⑤), which indicates the data node. Lastly, the background thread writes the key (⑥). We add the *mfence* instruction to guarantee the ordering of the store instruction, a practice consistent with previous studies [17,18,38,40].

Note that the background threads do not delete the entry or decrease the index number, which is stored as a value of the entry.

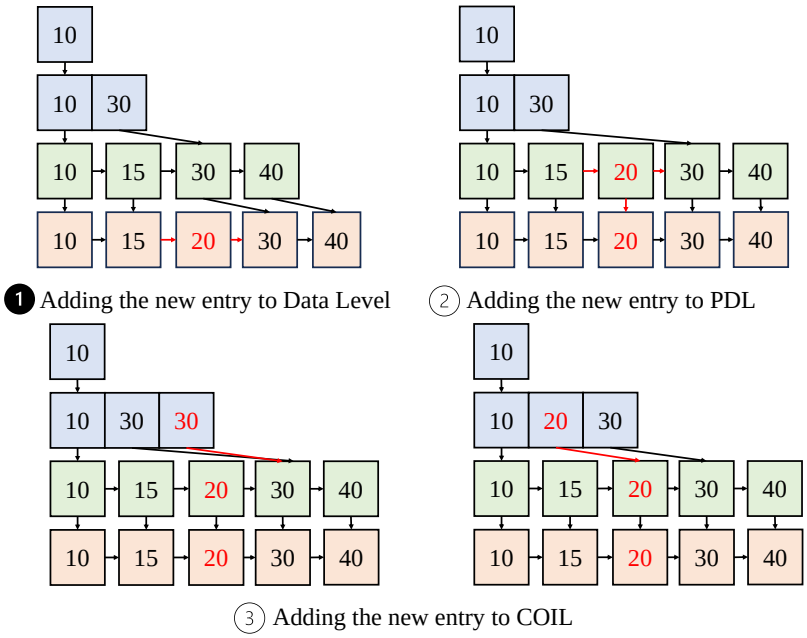


Figure 5. An illustration of ESL's async update. The foreground thread inserts new data into the data level ❶; a background thread inserts the data to the PDL ② and the COIL ③.

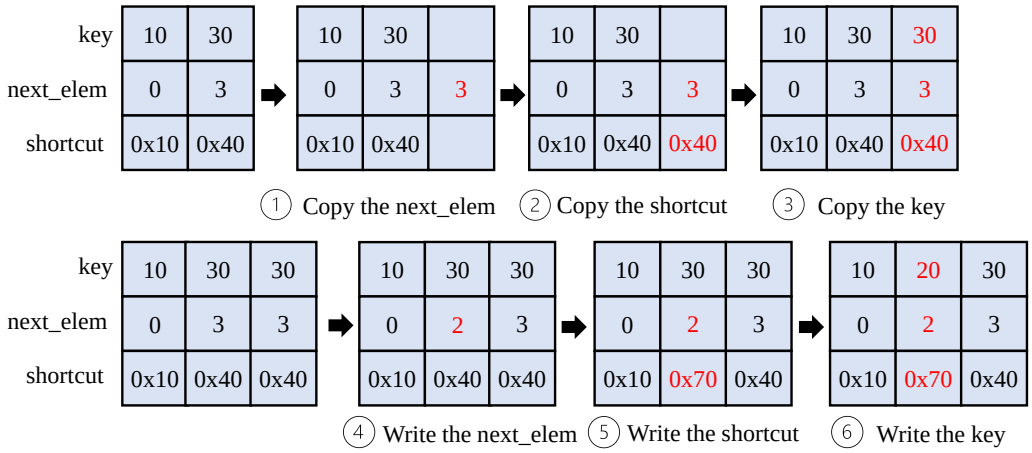


Figure 6. An illustration of the shift operation. ESL moves the next_elem, the shortcut, and the key in order.

4.4. Non-Blocking Read

ESL's read operation is composed of three phases: (1) Finding the parent node of the target data node or its neighboring node from the COIL. (2) Finding the target data node or its neighboring node from the PDL. (3) Finding the target data node from the data level. The read operation of ESL is trivial; however, there can be inconsistent states at each level because the background threads may update the index levels asynchronously. ESL tolerates these inconsistent states, as follows:

Tolerating Inconsistencies in the COIL. In the read operation, the reader traverses the index levels. To accelerate the performance, ESL leverages an exponential search to skip the unnecessary key comparison. When the exponential search reduces the traverse range, the reader searches for the proper key using a linear search. In the search operation, the reader first compares the key of the index level and then moves to the next data until the reader finds the same key as the given key. In this case, the reader moves to the data node at the data level directly. When the reader reaches the key larger than the given key, it moves to the next index level.

In ESL's insert operation (see Section 4.2), within the COIL, keys, values, and shortcuts are shifted in the same direction as the read operation. The key is updated last, as shown in Figure 6. Hence, the reader will not encounter a key larger than the specified key. If a reader arrives at the node adjacent to the target node, they will traverse further to locate the desired node.

ESL's delete operation (Section 4.2) in the COIL does not shift the data in the index levels. Instead, it logically deletes the key–value pair by setting the value to 0 or NULL. Hence, there is no shift operation in the delete operation.

Tolerating Inconsistencies in the PDL/DL. The read operation at the data level or PDL is intuitive. Since the PDL and DL are lock-free linked lists, the linked list node, which is being created, is not visible to the reader until it is connected by the *next pointer*. Hence, the reader can find a consistent state only.

Tolerating Inconsistencies between the COIL and the PDL/DL. Since the index levels and the PDL/DL are decoupled, the reader will reach another node, which is the neighbor of the target node. Similar to that of the COIL, the reader traverses the PDL or DL to find the target node.

4.5. Concurrency Control

4.5.1. Version-Based Locking Protocol for Updating the COIL

In the COIL, ESL leverages the version-based locking protocol to prevent the write/write conflict for updating the COIL. Since ESL allows, at most, one writer to increase the

version value, when the version number is odd, other writers wait until the version number becomes even. The version number is updated using atomic instructions. Note that the read operation in the COIL does not acquire the read lock as ESL employs the ROWEX protocol.

4.5.2. Lock-Free Operation Log

ESL uses the lock-free operation log to hand over the work from the foreground thread to the background thread. Figure 7 illustrates how the operation log works with multiple threads, guaranteeing atomicity using a CAS operation. The operation log features two variables that store the index values: one for the head and the other for the tail of the operation log. The head is used for multiple threads. Before processing the operation log, the background thread updates the head of the operation log atomically. The background thread can process only when the head is successfully updated. The foreground thread updates the tail of the operation log atomically when it writes the operation log entry to the operation log. The operation log entry consists of the operation, the key–value pair, the path, and the level. The path information includes the entries in the index level and the PDL that the writer visited.

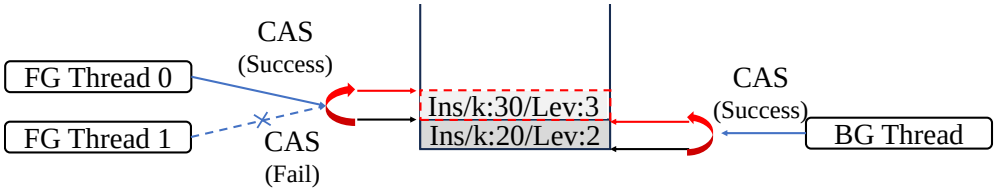


Figure 7. An illustration of the operation log. A head is updated by the background threads and a tail is updated by the foreground threads.

5. Implementation

We implement ESL using C++ and use the *xorshift random number generator* [43] to generate the level value of each node randomly.

6. Evaluation

6.1. Experimental Environment

Hardware. We performed experiments on a machine equipped with two NUMA sockets, each featuring an Intel Xeon Gold 5318Y processor with 24 physical cores. The machine offers a memory space of 768 GB DDR4 DRAM.

Workload. We used the YCSB workload [44], one of the representative workloads of key–value stores, as well as the Amazon AWS OpenStreetMap workload [45] to represent real-world scenarios for evaluating the performance of ESL.

In the YCSB evaluation, the YCSB workload consists of four workloads, workloads A–D. Workload A is composed of 50% of the update operations and 50% of the read operations. Workload B and workload D are composed of 5% of update operations and 95% of read operations. Workload C is composed of 100% of the read workload. Note that we did not include the results of the workload E evaluation, as the competitors did not implement the scan operation.

In the real-world workloads, we used three datasets, denoted as *longitudes*, *longlat*, and *lognormal*. The workloads are composed of the combinations of read and insert operations. We conducted two types of workloads—the read-intensive workload and write-intensive workload—similar to the previous study [34]. The read-intensive workload consists of 95% of read operations and 5% of insert operations. In the write-intensive workload, the ratio of the read operations and the write operations is the same (i.e., 50% read operations and 50% insert operations).

In each experiment, we inserted 1 million 8-byte keys and 8-byte values in the load phase. After inserting the key–value pairs, we conducted the evaluation with 1 million operations. For the default settings of competitors, the maximum level value was $\log_2(number_of_data)$ [46,47]. Similar to previous work [26,27,46], we used the default

maximum level of 19 (i.e., $\log_2(1Million)$). We also evaluated the maximum level of 5. We denoted the maximum level of 19 as the high level and the maximum level of 5 as the low level.

Competitors We compared the performance of ESL against other skiplist data structures. We included the No Hot Spot Skiplist (NHSSL) [26] and NUMASK [27]. We used the publicly available version of NHSSL and NUMASK in Synchrobench [46,47]. We could not evaluate the Cache-Sensitive Skiplist (CSSL) [24] because the open-sourced version did not consider the multi-thread and randomized key insertion. The implementation of the Parallel in-memory Skiplist (PSL) [25] was not open-sourced as of writing this paper.

6.2. Throughput

6.2.1. High Level (Maximum Level 19)

We conducted the performance evaluation using the YCSB workload. In this experiment, we set the maximum level value to 19 and used multiple threads, reaching up to 48. ESL outperformed other skiplists by up to 2.8× for all YCSB workloads. This corroborates our design goal of reducing cache misses and guaranteeing scalable access in a lock-free manner. Interestingly, in our evaluation, NHSSL demonstrated superior performance when compared to NUMASK, as shown in Figure 8. We analyzed the elapsed time for the query in Section 6.4.

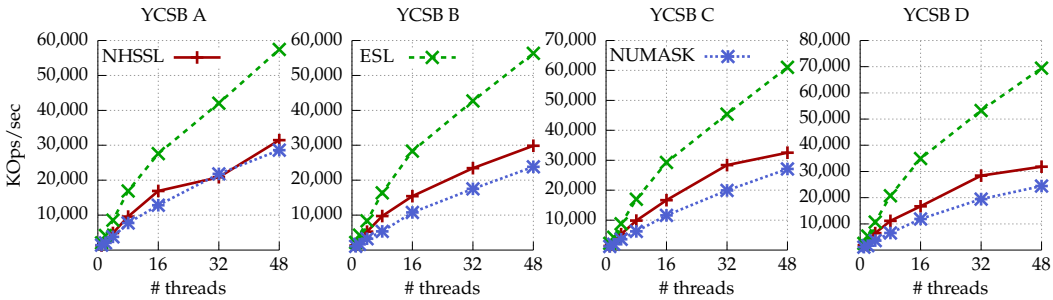


Figure 8. Performance comparison of the skiplists for the YCSB workload (level 19).

6.2.2. Low Level (Maximum Level 5)

We evaluated the skiplists with a maximum level of 5. Figure 9 presents the skiplist performances with a maximum level of 5. In the lower level, each skiplist level has more data. In general, the index performances were degraded because they lost the opportunity to skip keys as the level was low. ESL shows a superior read performance due to the COIL with an exponential search. Also, the lock-free linked list-based PDL and the data level reduced the synchronization overhead. The experimental results show that all workloads, including workloads A–D, had similar performance trends, as shown in Figure 9. ESL showed a performance enhancement of up to 19.9×, attributable to its cache-optimized COIL, the lock-free linked list-based data level, and the PDL. When the number of threads increased from 16 to 32, the performance improvement of ESL and NHSSL moderated because of the NUMA effect. Since the data number at each level increased, the NUMA remote latency had more of an impact on the ESL and NHSSL performances. NUMASK uses NUMA local index levels, resulting in fewer NUMA remote accesses compared to ESL and NHSSL.

As the data levels of all the skiplists in the evaluation used a lock-free linked list, the contention was not that high, even with the update workload. Also, the skiplist's update operation in this paper only changed the pointer value in the data node, so it did not trigger the SMO.

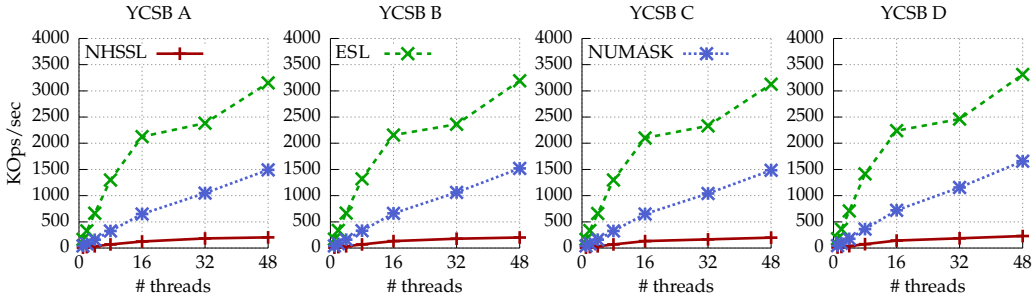


Figure 9. Performance comparison of the skiplists for the YCSB workload (level 5).

6.3. Tail Latency

6.3.1. High Level (Maximum Level 19)

In this experiment, we configured the skiplists to have a maximum level of 19 and used 48 threads, as shown in Figure 10. ESL has the lowest latency, both in terms of median and tail latency. The COIL of ESL efficiently reduces the traverse time and minimizes synchronization overhead during traversal. ESL achieves up to 35× shorter latency than other skiplists in the YCSB workload. NHSSL and NUMASK leverage the lock-free linked list-based level, so they do not have significant synchronization overhead, but they suffer from performance degradation from the cache misses that are inherited from the linked list. NUMASK shows lower tail latency than NHSSL because of its index-level replication for each NUMA socket.

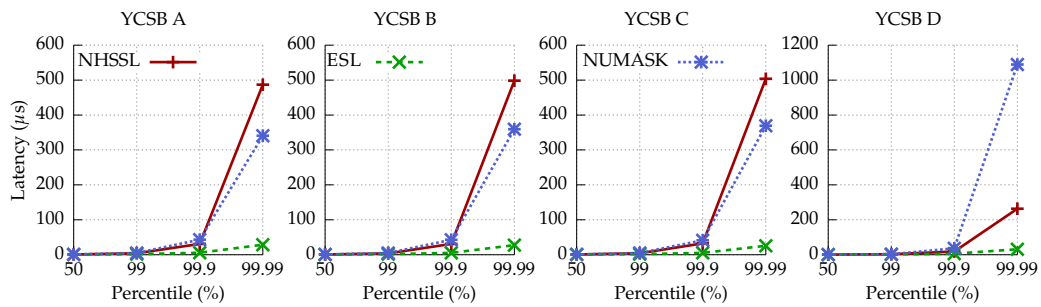


Figure 10. Tail latency comparison (Level 19).

6.3.2. Low Level (Maximum Level 5)

Figure 11 shows the tail latency of a skiplist when the maximum level is 5 and the number of threads is 48. Since the level is lower than the previous experiment, the number of nodes in each level is increased. Hence, the tail latencies of indices are increased. NHSSL shows the highest latency because the number of linked list traverse operations increased. Moreover, NUMA remote accesses further contribute to the latency delay. NUMASK shows better tail latency as it reduces the number of NUMA remote accesses by replicating the index levels. ESL shows short tail latency in our evaluation thanks to the COIL and lock-free linked list-based PDL and the data level.

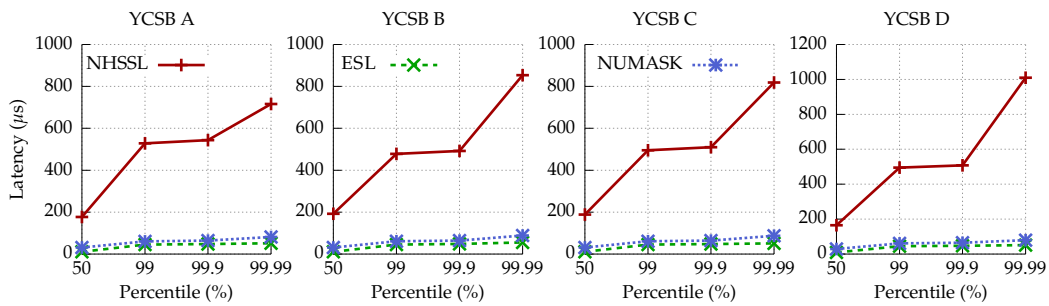


Figure 11. Tail Latency comparison (Level 5).

6.4. Performance Breakdown

We analyzed the skiplist performances by measuring the elapsed time for each component in the skiplists. We conducted the YCSB workload using 48 threads in the experiment.

6.4.1. High Level (Maximum Level 19)

Figure 12a presents the elapsed time for each component in the skiplist. ESL and NHSSL spend the most time traversing index levels. The experimental results show that NUMASK uses more time traversing the data level than NHSSL. This causes performance degradation, as described in Section 6.2. Note that traversing the PDL overhead in ESL is trivial because its lock-free linked list eliminates a lot of shift operations, which are inevitable in the array data structure. Also, the elapsed time for traversing the data level in ESL is shorter than in the other skiplist.

6.4.2. Low Level (Maximum Level 5)

Figure 12b shows the performance breakdown of the skiplists when the maximum level is 5. For all skiplists, most of their time is spent traversing index levels as there are more data than in the high-level (maximum-level 19) case. ESL reduces the traversing index level time under NHSSL and NUMASK by 14.5× and 2.1×, respectively. In the low-level case, the NUMA remote accesses have a significant impact on the performance of traversing the index levels. Index levels of NHSSL consist of linked lists, so the NUMA remote access latency is reflected due to the cache misses. NUMASK replicates the search layer to each socket; it reduces the number of NUMA remote accesses, but it still suffers from the cache misses inherited from the linked list-based structure. The time for traversing the PDL and the data level is much shorter than the time for traversing index levels. We also include the elapsed time for traversing the PDL and the data level in Table 1.

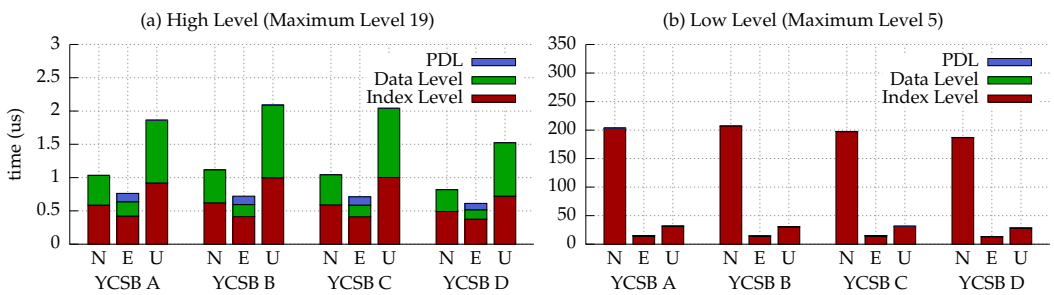


Figure 12. Performance breakdown for workloads A–D. In this experiment, N stands for NHSSL, E stands for ESL, U stands for NUMASK. Elapsed time for traversing the PDL and the data level is relatively short at a low level, so we added a table for presenting the time, as shown in Table 1.

Table 1. Elapsed time for traversing the PDL and the data level at a low level (µs). In this experiment, N stands for NHSSL; E stands for ESL; U stands for NUMASK.

		A	**B**	**C**	**D**
N	PDL	0	0	0	0
	Data level	0.25	0.28	0.26	0.21
E	PDL	0.17	0.16	0.16	0.14
	Data level	0.25	0.21	0.21	0.17
U	PDL	0	0	0	0
	Data level	0.42	0.41	0.40	0.30

6.5. Cache Miss

We measured the number of cache misses of the skiplists with YCSB workloads using *perf* [48]. We conducted tests on both low and high levels of the skiplists with 48 threads. As shown in Figure 13, ESL shows fewer cache misses than other skiplists because the COIL design is cache-friendly. ESL reduces the number of cache misses by up to 15.7×. All skiplists show fewer cache misses in the YCSB workload D because workload D reads the latest updated data. Interestingly, NUMASK shows more cache misses than NHSSL. This is because NUMASK leverages multiple search layers, and each search layer is stored on the corresponding NUMA memory. In this case, the search layer may not be in a consistent state, and it may lead to more linked list traverses, which increase the number of cache misses.

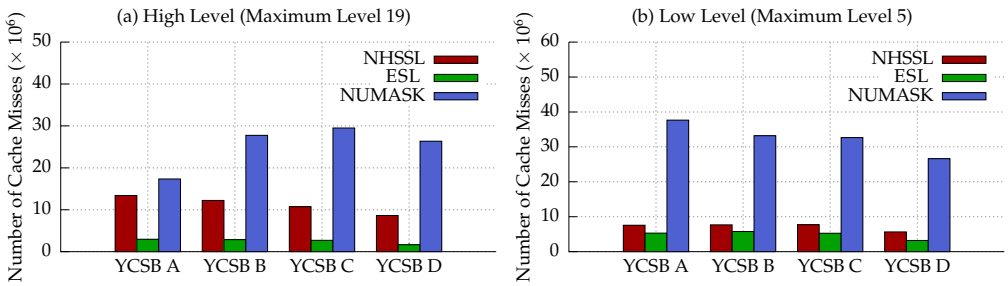


Figure 13. Number of cache misses of the skiplist with YCSB workloads.

6.6. Real-World Workload

We conducted the throughput evaluation with the AWS OpenStreetMap real-world workload, as mentioned in Section 6.1. We set the maximum level of skiplists to 19, as in the high level. The *longitude* dataset comprises 8-byte double-type data, representing longitude values. The *longlat* dataset is a set of 8-byte double keys. Each key is a combination of longitude and latitude. The *lognormal* dataset is a set of 8-byte integer keys with a lognormal distribution. We used 48 threads in this evaluation.

As shown in Figure 14, ESL exhibits lower throughput in both read-intensive and write-intensive workloads compared to YCSB workloads because the insert operation triggers an asynchronous update of the COIL, which may lead to dirty CPU cache lines. The COIL consists of consecutive arrays, so making dirty cache lines can degrade the performance. Despite the performance degradation, ESL consistently shows better performance than other skiplists. In the experiment with the read-intensive workload, ESL outperforms other skiplists because of its cache-friendly design. The ESL performance is degraded in the write-intensive workload because of the asynchronous update of the COIL.

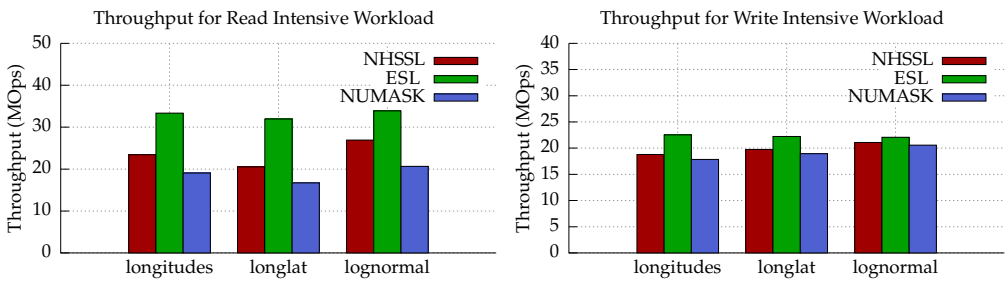


Figure 14. Throughput of the skiplists with the real-world workload.

7. Discussion

7.1. Limitations of ESL

Limitations of the asynchronous update. The COIL of the ESL shifts the key–value pairs to add a new one to each index level. Even though the ESL leverages asynchronous updates for the COIL and the PDL, the overhead of the shift operation can be too high when the

number of key–value pairs is too large. In that case, ESL can add more background threads or exploit a more lock-free linked list to the upper level of the PDL. Also, the asynchronous update makes the cache line dirty, so the traverse overhead of concurrent threads can be larger.

Cache misses when moving to the lower level. In the original skiplist, each node keeps the pointers of all levels. Hence, moving to the next level does not incur the cache misses. However, ESL merges the key–value pairs in the same level, so each level is separated. Thus, moving to the next level in ESL accompanies cache misses.

7.2. Skiplist for Emerging Storage Media

Since the skiplist is used as the key component of the storage system stack, there have been previous research efforts to improve the skiplists for emerging storage media, such as persistent memory. *The atomic skiplist (AS) and atomic and selective consistency skiplist* (ASCS) [49] are designed for persistent memory. They insert and delete an entry in order. Hence, they can guarantee data consistency without logging. Also, ASCS ensures the persistence of the lowest index level and the data level to reduce the persistence overhead. *The partitioned hierarchical skiplist (PhaST)* [50] is another skiplist designed for persistent memory. PhaST also stores data nodes at the data level in persistent memory and index levels in DRAM. Also, PhaST partitions the key range and uses another index cache to reduce the cache misses during skiplist traversal. *NoveLSM* [51] employs persistent skiplist-based MemTable to mitigate the serialization/deserialization overheads in the LSM tree-based key–value store. *ListDB* [52] merges the log of the key–value store with the persistent skiplist to reduce the redundant writes in the LSM-tree-based key–value store. The previous studies focus on reducing persistence overhead; however, ESL mainly targets volatile DRAM-based main memory systems. Also, ESL can be extended to the persistent version by storing the data layer in persistent memory.

8. Conclusions

We presented ESL, a high-performance skiplist. ESL uses arrays of nodes in the index level, denoted as the *COIL*, combined with exponential search to obtain high performance and scalability. In order to reduce the length of the critical section, ESL leverages asynchronous updates for updating the COIL. At the data level of ESL, we exploit a lock-free linked list to reduce synchronization overhead. We introduced the PDL, the lowest index level of the COIL, which employs a lock-free linked list to significantly minimize shift operations. In addition, we described how ESL's foreground threads tolerate the inconsistencies caused by background threads. In the performance evaluation, we compared the performance of ESL against *NHSSL* and *NUMASK*. The evaluation results show that ESL achieves up to 2.8× higher throughput than other in-memory skiplists in high-level evaluations. In the tail latency evaluation, ESL registers 35× lower tail latency compared to other skiplists.

Author Contributions: Conceptualization, methodology, Y.N., B.K. and W.-H.K.; software, Y.N., B.K. and T.P.; validation, Y.N., T.P., J.P. and W.-H.K.; writing—original draft preparation, Y.N. and W.-H.K.; writing—review and editing, Y.N., J.P. and W.-H.K.; supervision, W.-H.K.; project administration, W.-H.K.; funding acquisition, W.-H.K. All authors have read and agreed to the published version of the manuscript.

Funding: This paper was supported by Konkuk University in 2022.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data used in the paper is available from the corresponding author upon request.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Wang, T.; Johnson, R. Scalable Logging Through Emerging Non-volatile Memory. In Proceedings of the 40th International Conference on Very Large Data Bases (VLDB), Hangzhou, China, 1–5 September 2014.
2. Arulraj, J.; Perron, M.; Pavlo, A. Write-behind Logging. In Proceedings of the 42nd International Conference on Very Large Data Bases (VLDB), New Delhi, India, 5–9 September 2016.
3. Oh, G.; Kim, S.; Lee, S.W.; Moon, B. SQLite Optimization with Phase Change Memory for Mobile Applications. In Proceedings of the 41st International Conference on Very Large Data Bases (VLDB), Kohala Coast, HI, USA, 31 August–4 September 2015; pp. 1454–1465.
4. Park, J.H.; Oh, G.; Lee, S.W. SQL Statement Logging for Making SQLite Truly Lite. In Proceedings of the 43rd International Conference on Very Large Data Bases (VLDB), Munich, Germany, 28 August–1 September 2017; pp. 513–525.
5. Seo, J.; Kim, W.H.; Baek, W.; Nam, B.; Noh, S.H. Failure-Atomic Slotted Paging for Persistent Memory. In Proceedings of the 22nd ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Xi'an, China, 8–12 April 2017.
6. Kim, W.H.; Kim, J.; Baek, W.; Nam, B.; Won, Y. NVWAL: Exploiting NVRAM in Write-Ahead Logging. In Proceedings of the 21st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Atlanta, GA, USA, 2–6 April 2016.
7. Volume of Data/Information Created, Captured, Copied, and Consumed Worldwide from 2010 to 2020, with Forecasts from 2021 to 2025. Available online: https://www.statista.com/statistics/871513/worldwide-data-created/ (accessed on 28 August 2023).
8. DRAM and NAND Flash Prices Expected to Fall Further in 2Q23 Due to Weak Server Shipments and High Inventory Levels, Says TrendForce. Available online: https://www.trendforce.com/presscenter/news/20230509-11667.html (accessed on 28 August 2023).
9. Anandtech. Intel Launches Optane DIMMs Up To 512GB: Apache Pass Is Here! 2018. Available online: https://www.anandtech.com/show/12828/intel-launches-optane-dimms-up-to-512gb-apache-pass-is-here (accessed on 6 June 2023).
10. CXL Consortium. Compute Express Link: The Breakthrough CPU-to-Device Interconnect 2020. Available online: https://www.computeexpresslink.org/ (accessed on 1 July 2023).
11. Kozyrakis, C.; Kansal, A.; Sankar, S.; Vaid, K. Server Engineering Insights for Large-Scale Online Services. *IEEE Micro* **2010**, *30*, 8–19. [CrossRef]
12. Spec CPU2016 Results. Available online: https://www.spec.org/cpu2006/results/ (accessed on 28 August 2023).
13. Spec CPU2017 Results. Available online: https://www.spec.org/cpu2017/results/ (accessed on 28 August 2023).
14. 50 Years of Microprocessor Trend Data. Available online: https://github.com/karlrupp/microprocessor-trend-data (accessed on 28 August 2023).
15. Lehman, P.L.; Yao, S.B. Efficient Locking for Concurrent Operations on B-Trees. *ACM Trans. Database Syst.* **1981**, *6*, 650–670. [CrossRef]
16. Rao, J.; Ross, K.A. Making B+- Trees Cache Conscious in Main Memory. In Proceedings of the 2000 ACM SIGMOD/PODS Conference, Dallas, TX, USA, 16–18 May 2000; pp. 475–486.
17. Hwang, D.; Kim, W.H.; Won, Y.; Nam, B. Endurable Transient Inconsistency in Byte-addressable Persistent B+-tree. In Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST), Oakland, CA, USA, 12–15 February 2018; pp. 187–200.
18. Liu, J.; Chen, S.; Wang, L. LB+-Trees: Optimizing Persistent Index Performance on 3DXPoint Memory. *Proc. VLDB Endow.* **2020**, *13*, 1078–1090. [CrossRef]
19. Pugh, W. Skip Lists: A Probabilistic Alternative to Balanced Trees. *Commun. ACM* **1990**, *33*, 668–676. [CrossRef]
20. Google. LevelDB. Available online: https://github.com/google/leveldb (accessed on 20 July 2023).
21. Facebook. RocksDB. Available online: http://rocksdb.org/ (accessed on 20 July 2023).
22. Apache. Welcome to Apache HBase™. Available online: https://hbase.apache.org/ (accessed on 20 July 2023).
23. MongoDB. Available online: https://www.mongodb.org/ (accessed on 20 July 2023).
24. Sprenger, S.; Zeuch, S.; Leser, U. Cache-Sensitive Skip List: Efficient Range Queries on Modern CPUs. In *Data Management on New Hardware: 7th International Workshop on Accelerating Data Analysis and Data Management Systems Using Modern Processor and Storage Architectures, ADMS 2016 and 4th International Workshop on In-Memory Data Management and Analytics, IMDM 2016, New Delhi, India, 1 September 2016, Revised Selected Papers 4*; Blanas, S., Bordawekar, R., Lahiri, T., Levandoski, J.J., Pavlo, A., Eds.; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2016; Volume 10195, pp. 1–17. [CrossRef]
25. Xie, Z.; Cai, Q.; Jagadish, H.; Ooi, B.C.; Wong, W.F. Parallelizing Skip Lists for In-Memory Multi-Core Database Systems. In Proceedings of the 33rd IEEE International Conference on Data Engineering (ICDE), San Diego, CA, USA, 19–22 April 2017. [CrossRef]
26. Crain, T.; Gramoli, V.; Raynal, M. No Hot Spot Non-blocking Skip List. In Proceedings of the 33rd International Conference on Distributed Computing Systems (ICDCS), Philadelphia, PA, USA, 8–11 July 2013; pp. 196–205.
27. Daly, H.; Hassan, A.; Spear, M.F.; Palmieri, R. NUMASK: High Performance Scalable Skip List for NUMA. In Proceedings of the 31st International Conference on Distributed Computing (DISC), New Orleans, LA, USA, 16–20 October 2017; pp. 18:1–18:19.
28. Bentley, J.L.; Yao, A.C.C. An almost optimal algorithm for unbounded searching. *Inf. Process. Lett.* **1976**, *5*, 82–87. [CrossRef]
29. Kim, C.; Chhugani, J.; Satish, N.; Sedlar, E.; Nguyen, A.D.; Kaldewey, T.; Lee, V.W.; Brandt, S.A.; Dubey, P. FAST: Fast Architecture Sensitive Tree Search on Modern CPUs and GPUs. In Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data, New York, NY, USA, 6–10 June 2010; SIGMOD'10, pp. 339–350. [CrossRef]

30. Cha, H.; Hao, X.; Wang, T.; Zhang, H.; Akella, A.; Yu, X. Blink-hash: An Adaptive Hybrid Index for In-Memory Time-Series Databases. *Proc. VLDB Endow.* **2023**, *16*, 1235–1248. [CrossRef]
31. Choe, J.; Crotty, A.; Moreshet, T.; Herlihy, M.; Bahar, R.I. Hybrids: Cache-conscious concurrent data structures for near-memory processing architectures. In Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures, Philadelphia, PA, USA, 11–14 July 2022; pp. 321–332.
32. Kang, H.; Zhao, Y.; Blelloch, G.E.; Dhulipala, L.; Gu, Y.; McGuffey, C.; Gibbons, P.B. PIM-tree: A Skew-resistant Index for Processing-in-Memory. In Proceedings of the 2023 ACM Workshop on Highlights of Parallel Computing, New York, NY, USA, 16 June 2023; pp. 13–14.
33. Kraska, T.; Beutel, A.; Chi, E.H.; Dean, J.; Polyzotis, N. The Case for Learned Index Structures. In Proceedings of the 2018 International Conference on Management of Data, New York, NY, USA, 22–27 June 2018; SIGMOD'18, pp. 489–504. [CrossRef]
34. Ding, J.; Minhas, U.F.; Yu, J.; Wang, C.; Do, J.; Li, Y.; Zhang, H.; Chandramouli, B.; Gehrke, J.; Kossmann, D.; et al. ALEX: An Updatable Adaptive Learned Index. In Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, New York, NY, USA, 14–19 June 2020; SIGMOD'20, pp. 969–984. [CrossRef]
35. Zhang, Z.; Chu, Z.; Jin, P.; Luo, Y.; Xie, X.; Wan, S.; Luo, Y.; Wu, X.; Zou, P.; Zheng, C.; et al. PLIN: A Persistent Learned Index for Non-Volatile Memory with High Performance and Instant Recovery. *Proc. VLDB Endow.* **2022**, *16*, 243–255. [CrossRef]
36. Li, P.; Hua, Y.; Zuo, P.; Chen, Z.; Sheng, J. ROLEX: A Scalable RDMA-oriented Learned Key-Value Store for Disaggregated Memory Systems. In Proceedings of the 21st USENIX Conference on File and Storage Technologies (FAST 23), Santa Clara, CA, USA, 21–23 February 2023; pp. 99–114.
37. Mathew, A.; Min, C. HydraList: A Scalable In-Memory Index Using Asynchronous Updates and Partial Replication. In Proceedings of the 46th International Conference on Very Large Data Bases (VLDB), Tokyo, Japan, 31 August–4 September 2020.
38. Kim, W.H.; Krishnan, R.M.; Fu, X.; Kashyap, S.; Min, C. PACTree: A High Performance Persistent Range Index Using PAC Guidelines. In Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP), Virtual, 26–29 October 2021; pp. 424–439.
39. Leis, V.; Scheibner, F.; Kemper, A.; Neumann, T. The ART of Practical Synchronization. In Proceedings of the International Workshop on Data Management on New Hardware, San Francisco, CA, USA, 27 June 2016; pp. 3:1–3:8.
40. Chen, Y.; Lu, Y.; Fang, K.; Wang, Q.; Shu, J. UTree: A Persistent B+-Tree with Low Tail Latency. *Proc. VLDB Endow.* **2020**, *13*, 2634–2648. [CrossRef]
41. Hart, T.E.; McKenney, P.E.; Brown, A.D.; Walpole, J. Performance of Memory Reclamation for Lockless Synchronization. *J. Parallel Distrib. Comput.* **2007**, *67*, 1270–1285. [CrossRef]
42. McKenney, P.E. Structured Deferral: Synchronization via Procrastination. *ACM Queue* **1998**, *20*, 20–39. [CrossRef]
43. Marsaglia, G. Xorshift RNGs. *J. Stat. Softw.* **2003**, *8*, 1–6. [CrossRef]
44. Cooper, B.F.; Silberstein, A.; Tam, E.; Ramakrishnan, R.; Sears, R. Benchmarking Cloud Serving Systems with YCSB. In Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC), Indianapolis, IN, USA, 10–11 June 2010; pp. 143–154.
45. Amazon AWS OpenStreetMap. Available online: https://registry.opendata.aws/osm/ (accessed on 23 August 2023).
46. Gramoli, V. More Than You Ever Wanted to Know About Synchronization: Synchrobench, Measuring the Impact of the Synchronization on Concurrent Algorithms. In Proceedings of the 20th ACM Symposium on Principles and Practice of Parallel Programming (PPoPP), San Francisco, CA, USA, 7–11 February 2015; pp. 1–10.
47. Synchrobench. Available online: https://github.com/gramoli/synchrobench (accessed on 1 July 2023).
48. Linux Perf Wiki. Available online: https://perf.wiki.kernel.org/index.php/Main_Page (accessed on 23 August 2023).
49. Xiao, R.; Feng, D.; Hu, Y.; Wang, F.; Wei, X.; Zou, X.; Lei, M. Write-Optimized and Consistent Skiplists for Non-Volatile Memory. *IEEE Access* **2021**, *9*, 69850–69859. [CrossRef]
50. Li, Z.; Jiao, B.; He, S.; Yu, W. PhaST: Hierarchical Concurrent Log-Free Skip List for Persistent Memory. *IEEE Trans. Parallel Distrib. Syst.* **2022**, *33*, 3929–3941. [CrossRef]
51. Kannan, S.; Bhat, N.; Gavrilovska, A.; Arpaci-Dusseau, A.; Arpaci-Dusseau, R. Redesigning LSMs for Nonvolatile Memory with NoveLSM. In Proceedings of the 2018 USENIX Annual Technical Conference (ATC), Boston, MA, USA, 11–13 July 2018.
52. Kim, W.; Park, C.; Kim, D.; Park, H.; ri Choi, Y.; Sussman, A.; Nam, B. ListDB: Union of Write-Ahead Logs and Persistent SkipLists for Incremental Checkpointing on Persistent Memory. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), Carlsbad, CA, USA, 11–13 July 2022; pp. 161–177.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.